

Question 2

Abstract Classes, Multiple Interface Implementation, Interface Inheritance, Encapsulation, and Default Field Values

Theory Component

Explore the boundaries between abstract classes and interfaces, focusing on code reuse and type definition.

1. **Abstract class vs. interface:** Compare how an abstract class and an interface handle (a) instance fields, (b) constructors, (c) method implementations, and (d) **encapsulation** (private fields with public accessors). Why can an interface have only `public static final` fields, while an abstract class can have private instance state?
2. **Multiple interface implementation and inheritance among interfaces:** Define a scenario where two base interfaces declare methods with the same signature, and a third interface extends both. What happens when a concrete class implements the third interface? Investigate the JLS rules for inheriting abstract methods with identical signatures from multiple superinterfaces. Does the implementing class satisfy both contracts with a single method? Support your answer with a small example.
3. **Default values and constructor execution order:** Examine the order of initialization when an object is created: (a) default values for instance fields (including reference types), (b) instance initializers, (c) constructor body. How does this guarantee that an uninitialised reference field is `null` before the constructor runs?

Code Component

Design and implement a small hierarchy that consolidates the theoretical points. Follow this specification:

- **Abstract class** `Vehicle`:
 - Private `String` field `model`.
 - Constructor that takes `model` and initialises it.
 - Public getter for `model`.
 - `abstract void start();`
 - Concrete method `void displayInfo()` that prints the model.
 - Do **not** initialise a declared `String` field `owner` (leave it default).
- **Interface** `GPS`:
 - `String getLocation();`
- **Interface** `AutoPilot` that **extends** `GPS` (interface inheritance):
 - `void engageAutoPilot();`

- **Class** `Car` that **extends** `Vehicle` and **implements** `AutoPilot`:
 - Private `int` field `speed`.
 - Constructor that sets `model` (via `super`) and initialises `speed` to 0.
 - Override `start()` to print a message.
 - Implement `getLocation()` to return a dummy location string.
 - Implement `engageAutoPilot()` to (i) increase `speed` by 10 using a **shorthand operator**, (ii) print a message containing the new `speed`.
 - Override `toString()` to return `"Car: " + model + " speed=" + speed`.

After implementing, write a `main` method that:

1. **Object creation and reference types:** Create a `Car` object. Assign it to a `Vehicle` reference, a `GPS` reference, and an `AutoPilot` reference.
2. **Member access and casting:** From the `Vehicle` reference, call `displayInfo()`. Attempt to call `start()` (why is this allowed without casting?). Try to call `engageAutoPilot()` and explain why a cast is required. Perform a safe downcast to `AutoPilot` and invoke the method.
3. **Shorthand operator effect:** Call `engageAutoPilot()` multiple times and observe the `speed` value using `toString()`. Show how `speed += 10` behaves inside the method.
4. **String concatenation and `toString()`:** Use the `+=` operator on a `String` to append the `Car` object and print the result. Explain how Java implicitly invokes `toString()`.
5. **Default value of reference field:** Print the value of `owner` for a `Car` instance. What is displayed, and why? Tie this to your theoretical research on default values and constructor execution order.
6. **Interface inheritance verification:** Using the `AutoPilot` reference, call `getLocation()` to prove that `Car` inherits the method requirement from `GPS` through `AutoPilot`. Explain why this works even though `Car` only explicitly states `implements AutoPilot`.

In your theoretical write-up, explicitly map each code behaviour to the relevant JLS rules or object-oriented principle you researched.